

EECS445 Project 2 Report

Shaotian Sun

May 7, 2026

Abstract

The project 2 of EECS445 (WN25) is **Paco's Puppy Problem**: *An exploration of deep learning techniques for classification and representation learning*. The yellow parts (required written answers) are presented in this report.

(This part is named as "abstract" to align the section number with P2 Spec.)

1 Data Preprocessing

- (a) i)
 - **Question:** What is the mean and standard deviation of each color channel (RGB), learned from the entire training partition?
 - **Answer:** The **means** for RGB are **123.824, 118.349, 94.37** separately; the **standard deviations** for RGB are **62.038, 58.634, 61.257**.
- (a) ii)
 - **Question:** Why do we extract the mean and standard deviation from the training set as opposed to the other data partitions?
 - **Answer:** The reason is that only the training set is the data that we are **assumed to know** when doing prediction. Including the validation/test set will lead to **overfitting** also, making the prediction (classification) weaker.
- (b)
 - See Figure [1](#).



Figure 1: Figure of 1b

2 Convolutional Neural Networks

- (a)

- **Question:** How many learnable float-valued parameters does the model have?

- **Answer:** We calculate the number of learnable float-valued parameters layer by layer.

Layer 0: 0;

Layer 1:

$$16 \times (5 \times 5 \times 3 + 1) = 1216; \quad (1)$$

Layer 2: 0;

Layer 3:

$$64 \times (5 \times 5 \times 16 + 1) = 25664; \quad (2)$$

Layer 4: 0;

Layer 5:

$$8 \times (5 \times 5 \times 64 + 1) = 12808; \quad (3)$$

Layer 6:

$$(32 + 1) \times 2 = 66; \quad (4)$$

Overall:

$$1216 + 25664 + 12808 + 66 = \mathbf{39754}; \quad (5)$$

- (f) i)

- **Question:** Describe at least two sources of noise that contribute to this behavior.

– **Answer:**

1. The **high learning rate** makes the update step too big to decrease validation error (also training error).
2. **Overfitting** to the training set may lead to increase of validation error too.
3. Another possible source of noise is the **noises that exists in the data**, e.g. outlier images.

• (f) ii)

– **Question:** 1. Report the epoch. 2. Answer: What epoch does your model stop training at now? Based on the training graphs, which patience value works better and why? In what scenario might increased patience be better? 3. Include the training plot.

– **Answer:**

1. The epoch is **16** when patience is 5; the epoch is **21** when patience is 10.
2. (a) The epoch is **21** when patience is 10.
 (b) Patience value **10** works better since the graphs show when patience=10, the final **validation accuracy, AUROC are greater, and the validation loss is smaller**.
 (c) The increased patience will be better **when the model is complex, the data has a high variance, and it is hard to converge**.
3. The training plots are Figure 2 and 3.

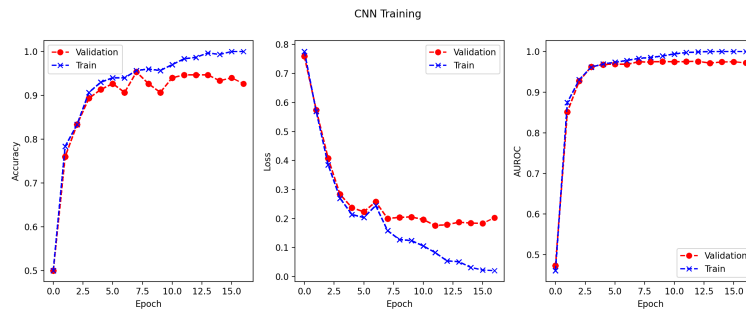


Figure 2: Training plot when patience = 5.

• (f) iii)

– **Question:** 1. What's the new size of the input to the fully connected layer? Report it. 2. Complete the table. 3. Answer: How does performance vary as we increase the number of filters from 8 to 64? What might account for this change?

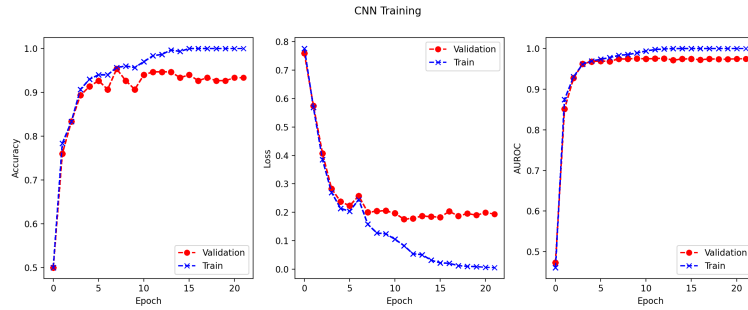


Figure 3: Training plot when patience = 10.

– **Answer:**

1. The new input size is

$$64 \times 2 \times 2 = \mathbf{256}.$$

2. See Table 1.

	Epoch	Training AUROC	Validation AUROC
8 filters	11	0.9973	0.9753
64 filters	10	0.9999	0.9668

Table 1: Performance comparison of training and validation AUROC for different filter sizes.

3. (a) The performance based on validation AUROC becomes **worse when changing from 8 to 64**.
- (b) The possible reasons for that change includes **overfitting, vanishing gradients** as the model becomes more complex when increasing the filter size.

- (g) i)

- **Question:** Include your CNN’s scores in the table.
- **Answer:** See Table 2

	Training	Validation	Testing
Accuracy	0.9833	0.9467	0.66
AUROC	0.9973	0.9753	0.7484

Table 2: CNN’s scores.

- (g) ii)

- **Question:** Compare the training and validation performance. Do we see any evidence of overfitting here?
 - **Answer: No.** The metrics of their performance including accuracy and AUROC are close to each other. The training ones only exceed a little bit than validation ones.
- (g) iii)
 - **Question:** What do you notice about the images of the positive labeled data compared to the negative labeled data? Then, compare the validation and test performance. What trend do you see here and how could it be related to the data? Hypothesize a possible explanation for such a trend.
 - **Answer:** The images of the positive labeled data **have greener backgrounds including grass and trees** than those of the negative labeled ones. Based on comparison, the **test performance is much worse** than the validation performance. The trend is that the model is **overfitted to classify images based on greenness of the background**. The corresponding explanation is that:

In training and validation sets, images of Collies usually have a green background, while the images of Golden Retrievers rarely have one. Then, our model is trained to overfit over the green background feature. However, in the testing set, maybe small portion of images of Collies have green backgrounds, or the majority of images of Golden Retrievers have green backgrounds. That makes the testing performance much worse than validation one.

3 Transfer Learning

- (c)
 - **Question:** Include the final training plot. Report the epoch number.
 - **Answer:** The plot is shown in Figure 4. The corresponding epoch number is 10.

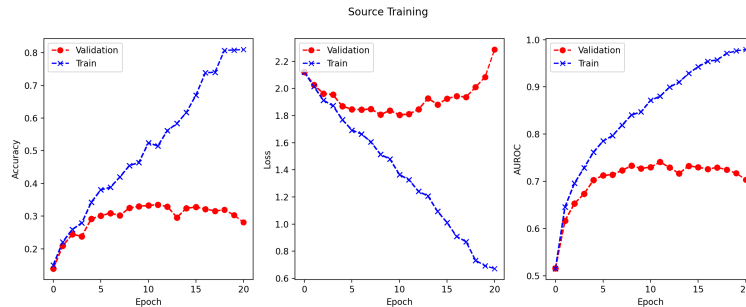


Figure 4: Figure of 3c.

- (d)
 - **Question:** Include the plot in your write up. Examine for which breed(s) the classifier is the most accurate. For which breed(s) is the classifier the least accurate? Briefly hypothesize why this might be the case.
 - **Answer:** The plot is shown in Figure 5. The most accurate breed is **Samoyed**; the most inaccurate breed is **Great Dane**. The brief hypothesis that I make is that **Samoyed might have some distinct features from all other breeds, but Great Dane shares a lot common features with other breeds.**
- (f)
 - **Question:**
 1. Compare this classifier to your previous CNN classifier in terms of the train, validation and test AUROC for the model associated with the lowest validation loss.
 2. Examine the results of using transfer learning and answer the following questions. Compared to the CNN in Section 2(g) that did not leverage the source task data, does transfer learning help? Was the source task helpful? Why do you hypothesize it was (or wasn't)?
 3. Examine the training curves. How does freezing all convolutional layers compare to freezing just a subset, versus freezing none? Why do you think this might occur?

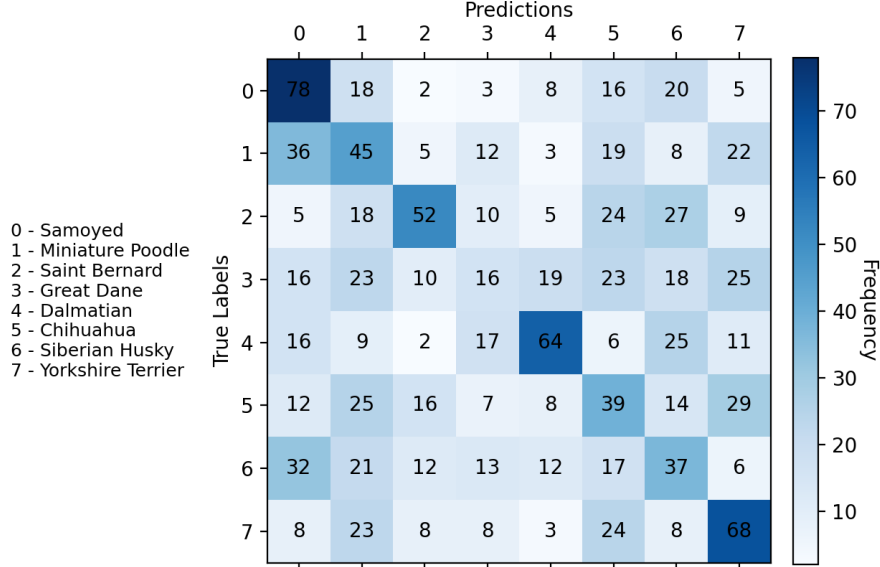


Figure 5: Figure of **3d**.

– **Answer:**

1. See Table [3](#).

	TRAIN	VAL	TEST
Freeze all CONV layers (Fine-tune FC layer)	0.8663	0.866	0.7124
Freeze first two CONV layers (Fine-tune last CONV and FC layers)	0.9956	0.965	0.7708
Freeze first CONV layer (Fine-tune last 2 conv. and fc layers)	0.9999	0.9655	0.7844
Freeze no layers (Fine-tune all layers)	1.0	0.9732	0.742
No Pretraining or Transfer Learning (Section 2(g) performance)	0.9753	0.9973	0.7484

Table 3: AUROCs for different frozen layers in transfer learning.

2. **Yes, transfer learning helps.** The source task was **helpful**. The reason I hypothesize so is that **the performance of freezing 1 or 2 convolutional layers based on AUROC is better than not leveraging transfer learning**.
3. Freezing all convolutional layers will need **much more epochs to converge** than freezing a subset or none. The training curve is **smoother when freezing more layers**.

4 Vision Transformers

- (a)
 - **Question:** Without such a learnable embedding how could you use the output of the transformer encoder to generate a classification? Explain your reasoning.
 - **Answer:** Without the learnable embedding, we could still use **average pooling** of the output to generate a classification.
REASONING: As stated in Section D.3 of *Dosovitskiy et al., 2021*, the embedding won't exceed other methods if using appropriate learning rates. Thus, the average pooling will still leverage the outstanding feature of the image no matter including this embedding or not.
- (b)
 - **Question:** What effect would this have on the model's ability to classify images? Explain your reasoning.
 - **Answer:** It will make the model's ability to classify images **worse**.
Explanation: **Without the information of position, the model cannot distinguish between pictures with similar small segments but different permutation of these segments, making it perform worse.**
- (f) i)
 - **Question:** How many learnable parameters do we need for the [cls] token?
 - **Answer:** **16**, since the number of learnable parameters is the dimension, which is $d=16$.
- (f) ii)
 - **Question:** How many learnable parameters are there involved with the Patch Linear Projection Layer and Position Embedding (excluding the [cls] token)?
 - **Answer:** Since the Patch Linear Projection Layer maps from a $3P^2$ -dimensional vector to a d -dimensional vector, the number of learnable parameters is:

$$(3P^2 + 1) \times d = (3 \times 4 \times 4 + 1) \times 16 = 784. \quad (6)$$

For Position Embedding (excluding the [cls] token), it's fixed, so there are no learnable parameters.

To conclude, there are overall **784**.

- (f) iii)

- **Question:** 1. How many learnable parameters are there in a single LayerNorm layer? 2. How many learnable parameters are there in a single MHA layer? 3. How many learnable parameters are there in a single MLP layer?

- **Answer:**

1. The learnable parameters include β and γ , i.e. 2 each dimension. Thus, it sums up to

$$2 \times 16 = 32. \quad (7)$$

To conclude, there are **32** learnable parameters in LayerNorm layer.

2. For each matrix Q, K, V , the dimension is

$$(16 + 1) \times (16/2) = 136. \quad (8)$$

In a single MHA layer, there are $2(\text{number of heads}) \times 3(Q, K, V) \times 136 = 816$ in the attention layer; there are $(16+1) \times 16 = 272$ in the linear layer (W^O).

To conclude, there are $816 + 272 = \mathbf{1088}$.

3. In a single MLP layer, the number of parameters of 2 MLP layers is:

$$\begin{aligned} & (d + 1) \times 4d + (4d + 1) \times d \\ &= (16 + 1) \times (4 \times 16) + (4 \times 16 + 1) \times 16 \\ &= 1088 + 1040 \\ &= \mathbf{2128}. \end{aligned}$$

- (f) iv)

- **Question:** How many learnable parameters are there in the final MLP head?

- **Answer:** There are overall

$$(16 + 1) \times 2 = \mathbf{34}. \quad (9)$$

- (f) v)

- **Question:** Using your previous answers and Appendix D, how many total learnable parameters are there in our ViT?

- **Answer:** There are overall

$$16 + 784 + 2 \times (32 \times 2 + 1088 + 2128) + 34 = \mathbf{7394}. \quad (10)$$

(NOTE: The "2" stands for the number of transformer blocks.)

• (h)

- **Question:** Be sure to include the training figures generated by the script in your writeup. Report the train/validation AUROC performance of the model from the epoch demonstrating the lowest validation loss.
- **Answer:** The training figures are shown in Figure 6. The train AUROC performance of the model is **0.9572**; the validation AUROC is **0.9659**.

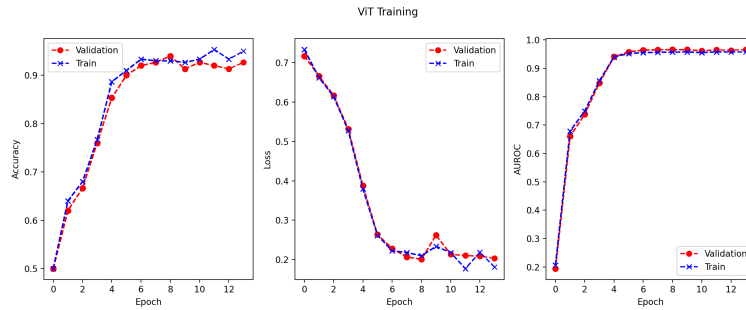


Figure 6: Figure of 4h.

• (i)

- **Question:** Report the saved model's AUROC performance on the test set.
- **Answer:** The test AUROC performance of the model is **0.5524**;

• (j)

- **Question:** How does the chosen ViT model's validation and test performance compare to the best chosen CNN validation and test performance on the target task from Section 2(g)? How does the number of learnable parameters in the ViT compare to the number of learnable parameters in the CNN? Explain why you believe they do or do not differ.
- **Answer:** The comparison between ViT and CNN from 2(g) is shown in Table 4. They do differ: *a*) the validation/test performance of ViT is **worse** than CNN from 2(g); *b*) the number of learnable parameters of ViT is **much fewer** than that of CNN from 2(g).

The explanation is that **with much more parameters, CNN might thus performs better in validation/test than ViT**. The reason that I believe that they do differ is based on the data in Table 4.

	VAL	TEST	# of learnable parameters
ViT	0.9659	0.5524	7394
CNN from 2(g)	0.9753	0.7484	39754

Table 4: Comparison between ViT and CNN from 2(g)

5 Challenge

5.1 Transfer Learning & Model Evaluation

5.1.1 Overview

After examining all the test performance based on AUROC, I found transfer learning CNN had the best performance.

Therefore, I decided to build the challenge model based on the transfer learning CNN, where **the source is trained on all other 8 breeds**.

To evaluate the model’s performance, I will use **test/validation(only when no test data split) AUROC** to determine which model is the best.

5.1.2 Source & Initial Target Training

The training though is exactly the same, but using the validation AUROC as the metric, the best one is Epoch 11. Then, we will use that one to do transfer learning. After trying all levels of freezing layers, the best test AUROC one is 0.7944 when we froze 2 convolutional layers and fine tuning. That’s our new starting point for the challenge. We’ll edit the `train_target_challenge.py` (currently a copy of `train_target.py`) in the following part to optimize the performance.

5.2 Regularization

In fine tuning section, we found the model might have an issue of overfitting since it turned out to have a extremely large validation AUROC but poor test AUROC. Regularization might help to fix it.

5.2.1 Weight Decay

The target training model could use different weight decays as shown in Table 5. Based on the metric, the best one is **using small weight decay** 1×10^{-5} .

Weight Decay	BEST test AUROC
1×10^{-5}	0.7992
1×10^{-4}	0.7864
1×10^{-3}	0.7904
1×10^{-2}	0.7928

Table 5: AUROC for different weight decay values.

5.2.2 Dropout

We could also employ dropout to do the regularization. As I experimented with dropout probability 0.2, 0.3, 0.4, 0.5, the best is 0.8104 when $p = 0.4$.

5.3 Data Augmentation

5.3.1 Resizing

We will use the `resize()` as provided and see the training result. The best test AUROC is the same as what we have got before.

5.3.2 Other Data Augmentations

Other than resizing, there are many ways to change images to help the model learn better. **Flipping, rotating, or cropping images** can make them look different while keeping important details. **Changing brightness, contrast, or colors, adding slight blur** will also help maybe.

5.4 Model architecture

5.4.1 Batch Normalization

Batch normalization helps improve the training stability and convergence speed. However, when I apply it, the result is worse than original. Thus, I decide to not use it.

5.4.2 Other Modification: Model Depth, Additional FC Layer...

Other potential modifications include model depth, like increasing or decreasing filter numbers; adding another fully-connected layer after the last one, and so on.

5.5 Hyperparameters

In this subsection, I tried to adjust the hyperparameters, e.g. patience.

5.5.1 Patience

I tried patience 5, 10, 15. The patience 10, 15 won't improve the test performance. Actually, it just makes some more epochs.

5.5.2 Learning Rate

I tried learning rate as 1e-3, 1e-4, 1e-5, which has no apparent improvement.

5.6 Discussion & Conclusion

Future improvements can be done based on adjusting the source training before transfer learning, using some other model architectures similar to ViT. Also, splitting data in different ways will definitely improve the final performance.

To conclude, I eventually applied the one with weight decay $1e - 5$ and dropout probability $p = 0.4$, where test $AUROC = 0.8104$.